

DS HW SET 2-2025115112

1. Define an integer array. Declare an integer pointer & initialize it to point to first element of array. use pointer arithmetic to traverse the array and print the elements. Modify the elements of the array using pointer arithmetic.

Code:

```
#include <stdio.h>

int main()
{
    int n;

    printf("Enter number of elements: ");

    scanf("%d", &n);

    int arr[n];

    int *ptr = arr;

    printf("Enter %d integers: ", n);

    for(int i = 0; i < n; i++)
    {
        scanf("%d", ptr + i);
    }

    printf("Original elements: ");

    for(int i = 0; i < n; i++)
    {
        printf("%d ", *(ptr + i));

        *(ptr + i) += 5;
    }

    printf("\nModified elements (+5): ");

    for(int i = 0; i < n; i++)
    {
        printf("%d ", *(ptr + i));
    }

    printf("\n");

    return 0;
}
```

Output:

Enter number of elements: 3

Enter 3 integers: 10 20 30

Original elements: 10 20 30

Modified elements (+5): 15 25 35

2. Calculate string length using pointers.

Code:

```
#include <stdio.h>
```

```
int main()
{
    char str[100];
    printf("Enter a string: ");
    scanf("%99s", str);
    char *ptr = str;
    int length = 0;
    while(*ptr != '\0')
    {
        length++;
        ptr++;
    }
    printf("String length: %d\n", length);
    return 0;
}
```

Output

Enter a string: Programming

String length: 11

3. Write a function that calculates both the sum and product of two numbers. Return these two results using pointers passed as arguments to the fun.

Code:

```
#include <stdio.h>

void calc(int a, int b, int *sum, int *product)
{
    *sum = a + b;
    *product = a * b;
}

int main()
{
    int num1, num2, s, p;
    printf("Enter two integers: ");
    scanf("%d %d", &num1, &num2);
    calc(num1, num2, &s, &p);
    printf("Sum: %d\nProduct: %d\n", s, p);
    return 0;
}
```

Output

Enter two integers: 5 4

Sum: 9

Product: 20

4. Create an array of structures. use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access/modify members.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Data
{
    int value;
};
int main()
{
    int n;
    printf("Enter array size: ");
    scanf("%d", &n);

    struct Data *ptr = (struct Data*)malloc(n * sizeof(struct Data));

    for(int i = 0; i < n; i++)
    {
        printf("Enter value for element %d: ", i + 1);
        scanf("%d", &(ptr + i)->value);
    }

    printf("\nStored values:\n");
    for(int i = 0; i < n; i++) {
        printf("Element %d: %d\n", i + 1, (ptr + i)->value);
    }

    free(ptr);
    return 0;
}
```

Output

```
Enter array size: 3
Enter value for element 1: 10
Enter value for element 2: 20
Enter value for element 3: 30
```

```
Stored values:
Element 1: 10
Element 2: 20
Element 3: 30
```

5. Define two functions with same signature. Declare a function pointer and use the pointer to call both functions.

Code:

```
#include <stdio.h>
void greetMorning()
{
    printf("Good Morning!\n");
}
void greetEvening()
{
    printf("Good Evening!\n");
}
int main()
{
    int choice;
    void (*funcPtr)();
    printf("Choose greeting (1 for Morning, 2 for Evening): ");
    scanf("%d", &choice);
    if (choice == 1)
    {
        funcPtr = greetMorning;
    }
    else
    {
        funcPtr = greetEvening;
    }

    funcPtr();
    return 0;
}
```

Output

Choose greeting (1 for Morning, 2 for Evening): 1
Good Morning!

6. Declare an integer, a pointer to it and a double pointer. Print the values and addresses at each level.

Code:

```
#include <stdio.h>

int main()
{
    int var;
    printf("Enter an integer: ");
    scanf("%d", &var);
    int *ptr = &var;
    int **dptr = &ptr;

    printf("Level 1 (var) - Value: %d, Address: %p\n", var, (void*)&var);
    printf("Level 2 (ptr) - Value: %p, Address: %p, Deref: %d\n", (void*)ptr, (void*)&ptr, *ptr);
    printf("Level 3 (dptr) - Value: %p, Address: %p, Deref: %p, Double Deref: %d\n", (void*)dptr, (void*)&dptr, (void*)*dptr, **dptr);

    return 0;
}
```

Output

Enter an integer: 100

Level 1 (var) - Value: 100, Address: 0x7ffe1234

Level 2 (ptr) - Value: 0x7ffe1234, Address: 0x7ffe1238, Deref: 100

Level 3 (dptr) - Value: 0x7ffe1238, Address: 0x7ffe1240, Deref: 0x7ffe1234, Double Deref: 100

7. Sort an array of floating-point numbers using Selection sort

Code:

```
#include <stdio.h>

int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    float arr[n];
    printf("Enter %d float values: ", n);
    for(int i = 0; i < n; i++)
    {
        scanf("%f", &arr[i]);
    }

    for (int i = 0; i < n - 1; i++)
    {
        int min_idx = i;
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[min_idx])
            {
                min_idx = j;
            }
        }
        float temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%.1f ", arr[i]);
    printf("\n");

    return 0;
}
```

Output

```
Enter number of elements: 4
Enter 4 float values: 4.5 1.2 3.8 2.1
Sorted array: 1.2 2.1 3.8 4.5
```

8. Define a structure Student with id, name and grade, create an array of Student structures and sort them based on different fields.

Code:

```
#include <stdio.h>
```

```
struct Student
```

```
{
    int id;
    char name[20];
    char grade;
};
```

```
void sortById(struct Student arr[], int n)
```

```
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (arr[i].id > arr[j].id)
            {
                struct Student temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}
```

```
int main()
```

```
{
    int n;
    printf("Enter number of students: ");
    scanf("%d", &n);

    struct Student students[n];
    for(int i = 0; i < n; i++) {
        printf("Enter ID, Name, Grade for student %d: ", i + 1);
        scanf("%d %s %c", &students[i].id, students[i].name, &students[i].grade);
    }
}
```

```
sortById(students, n);
```

```
printf("\nSorted by ID:\n");
```

```
for (int i = 0; i < n; i++) {
    printf("ID: %d | Name: %s | Grade: %c\n", students[i].id, students[i].name, students[i].grade);
}
```

```
    return 0;  
}
```

Output

Enter number of students: 3

Enter ID, Name, Grade for student 1: 103 Charlie B

Enter ID, Name, Grade for student 2: 101 Alice A

Enter ID, Name, Grade for student 3: 102 Bob C

Sorted by ID:

ID: 101 | Name: Alice | Grade: A

ID: 102 | Name: Bob | Grade: C

ID: 103 | Name: Charlie | Grade: B

9. Sort a set of negative numbers using Radix Sort.

Code:

```
#include <stdio.h>
```

```
int getMax(int arr[], int n)
```

```
{
    int max = arr[0];
    for (int i = 1; i < n; i++)
    {
        if (arr[i] > max) max = arr[i];
    }
    return max;
}
```

```
void countingSort(int arr[], int n, int exp)
```

```
{
    int output[n];
    int count[10] = {0};

    for (int i = 0; i < n; i++)
    {
        count[(arr[i] / exp) % 10]++;
    }

    for (int i = 1; i < 10; i++)
    {
        count[i] += count[i - 1];
    }

    for (int i = n - 1; i >= 0; i--)
    {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    for (int i = 0; i < n; i++)
    {
        arr[i] = output[i];
    }
}
```

```
void radixSortNegative(int arr[], int n)
```

```
{
    for (int i = 0; i < n; i++)
    {
        arr[i] = -arr[i];
    }
}
```

```

int max = getMax(arr, n);

for (int exp = 1; max / exp > 0; exp *= 10)
{
    countingSort(arr, n, exp);
}

for (int i = 0; i < n / 2; i++)
{
    int temp = -arr[i];
    arr[i] = -arr[n - 1 - i];
    arr[n - 1 - i] = temp;
}
if (n % 2 != 0) {
    arr[n / 2] = -arr[n / 2];
}
}

int main()
{
    int n;
    printf("Enter number of negative elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d negative integers: ", n);
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    radixSortNegative(arr, n);

    printf("Sorted negative array: ");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}

```

Output

```

Enter number of negative elements: 5
Enter 5 negative integers: -170 -45 -75 -90 -802
Sorted negative array: -802 -170 -90 -75 -45

```

10. Write a C program to copy one array to another using pointers.

Code:

```
#include <stdio.h>

int main()
{
    int n;
    printf("Enter array size: ");
    scanf("%d", &n);

    int source[n], dest[n];
    printf("Enter %d elements: ", n);
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &source[i]);
    }

    int *ptrSrc = source;
    int *ptrDest = dest;

    for (int i = 0; i < n; i++)
    {
        *(ptrDest + i) = *(ptrSrc + i);
    }

    printf("Copied Array: ");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", dest[i]);
    }
    printf("\n");

    return 0;
}
```

Output

```
Enter array size: 5
Enter 5 elements: 10 20 30 40 50
Copied Array: 10 20 30 40 50
```